JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class MonthDay

java.lang.Object
    java.time.MonthDay

All Implemented Interfaces:

Serializable, Comparable<MonthDay>, TemporalAccessor, TemporalAdjuster

---

```
public final class MonthDay
extends Object
implements TemporalAccessor, TemporalAdjuster, Comparable<MonthDay>, Serializable
```

A month-day in the ISO-8601 calendar system, such as `--12-03`.

`MonthDay` is an immutable date-time object that represents the combination of a month and day-of-month. Any field that can be derived from a month and day, such as quarter-of-year, can be obtained.

This class does not store or represent a year, time or time-zone. For example, the value "December 3rd" can be stored in a `MonthDay`.

Since a `MonthDay` does not possess a year, the leap day of February 29th is considered valid.

This class implements `TemporalAccessor` rather than `Temporal`. This is because it is not possible to define whether February 29th is valid or not without external information, preventing the implementation of plus/minus. Related to this, `MonthDay` only provides access to query and set the fields `MONTH_OF_YEAR` and `DAY_OF_MONTH`.

The ISO-8601 calendar system is the modern civil calendar system used today in most of the world. It is equivalent to the proleptic Gregorian calendar system, in which today's rules for leap years are applied for all time. For most applications written today, the ISO-8601 rules are entirely suitable. However, any application that makes use of historical dates, and requires them to be accurate will find the ISO-8601 approach unsuitable.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail. The `equals` method should be used for comparisons.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Method Summary

All Methods | Static Methods

Modifier and Type	Method	Description
Temporal	**adjustInto**(Temporal temporal)	Adjusts the specified temporal object to have this month-day.
LocalDate	**atYear**(int year)	Combines this month-day with a year to create a LocalDate.
int	**compareTo**(MonthDay other)	Compares this month-day to another month-day.
boolean	**equals**(Object obj)	Checks if this month-day is equal to another month-day.
String	**format**(DateTimeFormatter forma	Formats this month-day using the specified formatter.
static MonthDay	**from**(TemporalAccessor tempor	Obtains an instance of MonthDay from a temporal object.
int	**get**(TemporalField field)	Gets the value of the specified field from this month-day as an int.
int	**getDayOfMonth**()	Gets the day-of-month field.
long	**getLong**(TemporalField field)	Gets the value of the specified field from this month-day as a long.
Month	**getMonth**()	Gets the month-of-year field using the Month enum.

int	**getMonthValue**()	Gets the month-of-year field from 1 to 12.
int	**hashCode**()	A hash code for this month-day.
boolean	**isAfter**(MonthDay other)	Checks if this month-day is after the specified month-day.
boolean	**isBefore**(MonthDay other)	Checks if this month-day is before the specified month-day.
boolean	**isSupported**(TemporalField field)	Checks if the specified field is supported.
boolean	**isValidYear**(int year)	Checks if the year is valid for this month-day.
static MonthDay	**now**()	Obtains the current month-day from the system clock in the default time-zone.
static MonthDay	**now**(Clock clock)	Obtains the current month-day from the specified clock.
static MonthDay	**now**(ZoneId zone)	Obtains the current month-day from the system clock in the specified time-zone.
static MonthDay	**of**(int month, int dayOfMonth)	Obtains an instance of MonthDay.
static MonthDay	**of**(Month month, int dayOfMonth)	Obtains an instance of MonthDay.
static MonthDay	**parse**(CharSequence text)	Obtains an instance of MonthDay from a text string such as --12-03.
static MonthDay	**parse**(CharSequence text, DateTimeFormatter format	Obtains an instance of MonthDay from a text

		string using a specific formatter.
<R> R	**query**(TemporalQuery<R> query)	Queries this month-day using the specified query.
ValueRange	**range**(TemporalField field)	Gets the range of valid values for the specified field.
String	**toString**()	Outputs this month-day as a String, such as --12-03.
MonthDay	**with**(Month month)	Returns a copy of this MonthDay with the month-of-year altered.
MonthDay	**withDayOfMonth**(int dayOfMonth)	Returns a copy of this MonthDay with the day-of-month altered.
MonthDay	**withMonth**(int month)	Returns a copy of this MonthDay with the month-of-year altered.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Details

now

```
public static MonthDay now()
```

Obtains
the current month-day from the system clock in the default time-zone.

This will query the system clock in the default time-zone to obtain the current month-day.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Returns:

the current month-day using the system clock and default time-zone, not null

now

```
public static MonthDay now(ZoneId zone)
```

Obtains the current month-day from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current month-day. Specifying the time-zone avoids dependence on the default time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Parameters:

`zone` - the zone ID to use, not null

Returns:

the current month-day using the system clock, not null

now

```
public static MonthDay now(Clock clock)
```

Obtains the current month-day from the specified clock.

This will query the specified clock to obtain the current month-day. Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using `dependency injection`.

Parameters:

`clock` - the clock to use, not null

Returns:

the current month-day, not null

of

```
public static MonthDay of(Month month,
                          int dayOfMonth)
```

Obtains an instance of `MonthDay`.

The day-of-month must be valid for the month within a leap year. Hence, for February, day 29 is valid.

For example, passing in April and day 31 will throw an exception, as there can never be April 31st in any year. By contrast, passing in February 29th is permitted, as that month-day can sometimes be valid.

Parameters:

`month` - the month-of-year to represent, not null

`dayOfMonth` - the day-of-month to represent, from 1 to 31

Returns:

the month-day, not null

Throws:

`DateTimeException` - if the value of any field is out of range, or if the day-of-month is invalid for the month

of

```
public static MonthDay of(int month,
                          int dayOfMonth)
```

Obtains an instance of `MonthDay`.

The day-of-month must be valid for the month within a leap year. Hence, for month 2 (February), day 29 is valid.

For example, passing in month 4 (April) and day 31 will throw an exception, as there can never be April 31st in any year. By contrast, passing in February 29th is permitted, as that month-day can sometimes be valid.

Parameters:

`month` - the month-of-year to represent, from 1 (January) to 12 (December)

`dayOfMonth` - the day-of-month to represent, from 1 to 31

Returns:

the month-day, not null

Throws:

`DateTimeException` - if the value of any field is out of range, or if the day-of-month is invalid for the month

from

```
public static MonthDay from(TemporalAccessor temporal)
```

Obtains an instance of `MonthDay` from a temporal object.

This obtains a month-day based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `MonthDay`.

The conversion extracts the `MONTH_OF_YEAR` and `DAY_OF_MONTH` fields. The extraction is only permitted if the temporal object has an ISO chronology, or can be converted to a `LocalDate`.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `MonthDay::from`.

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the month-day, not null

Throws:

`DateTimeException` - if unable to convert to a `MonthDay`

parse

```
public static MonthDay parse(CharSequence text)
```

Obtains an instance of `MonthDay` from a text string such as `--12-03`.

The string must represent a valid month-day. The format is `--MM-dd`.

Parameters:

`text` - the text to parse such as "--12-03", not null

Returns:

the parsed month-day, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed

parse

```
public static MonthDay parse(CharSequence text,
                             DateTimeFormatter formatter)
```

Obtains an instance of `MonthDay` from a text string using a specific formatter.

The text is parsed using the formatter, returning a month-day.

Parameters:

`text` - the text to parse, not null

`formatter` - the formatter to use, not null

Returns:

the parsed month-day, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this month-day can be queried for the specified field. If false, then calling the `range` and `get` methods will throw an exception.

If the field is a `ChronoField` then the query is implemented here. The supported fields are:

- `MONTH_OF_YEAR`
- `YEAR`

All other `ChronoField` instances will return false.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field` - the field to check, null returns false

Returns:

true if the field is supported on this month-day, false if not

range

```
public ValueRange range(TemporalField field)
```

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This month-day is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return appropriate range instances. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

`field` - the field to query the range for, not null

Returns:

the range of valid values for the field, not null

Throws:

`DateTimeException` - if the range for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

get

```
public int get(TemporalField field)
```

Gets the value of the specified field from this month-day as an `int`.

This queries this month-day for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this month-day. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`get` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

UnsupportedTemporalTypeException - if the field is not supported or the range of values exceeds an int

ArithmeticException - if numeric overflow occurs

getLong

```
public long getLong(TemporalField field)
```

Gets the value of the specified field from this month-day as a long.

This queries this month-day for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a ChronoField then the query is implemented here. The supported fields will return valid values based on this month-day. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

getLong in interface TemporalAccessor

Parameters:

field - the field to get, not null

Returns:

the value for the field

Throws:

DateTimeException - if a value for the field cannot be obtained

UnsupportedTemporalTypeException - if the field is not supported

ArithmeticException - if numeric overflow occurs

getMonthValue

```
public int getMonthValue()
```

Gets the month-of-year field from 1 to 12.

This method returns the month as an int from 1 to 12. Application code is frequently clearer if the enum Month is used by calling getMonth().

Returns:

the month-of-year, from 1 to 12

See Also:

`getMonth()`

getMonth

`public Month getMonth()`

Gets the month-of-year field using the `Month` enum.

This method returns the enum `Month` for the month. This avoids confusion as to what `int` values mean. If you need access to the primitive `int` value then the enum provides the `int value`.

Returns:

the month-of-year, not null

See Also:

`getMonthValue()`

getDayOfMonth

`public int getDayOfMonth()`

Gets the day-of-month field.

This method returns the primitive `int` value for the day-of-month.

Returns:

the day-of-month, from 1 to 31

isValidYear

`public boolean isValidYear(int year)`

Checks if the year is valid for this month-day.

This method checks whether this month and day and the input year form a valid date. This can only return false for February 29th.

Parameters:

`year` - the year to validate

Returns:

true if the year is valid for this month-day

See Also:

`Year.isValidMonthDay(MonthDay)`

withMonth

`public MonthDay withMonth(int month)`

Returns a copy of this `MonthDay` with the month-of-year altered.

This returns a month-day with the specified month. If the day-of-month is invalid for the specified month, the day will be adjusted to the last valid day-of-month.

This instance is immutable and unaffected by this method call.

Parameters:

`month` - the month-of-year to set in the returned month-day, from 1 (January) to 12 (December)

Returns:

a `MonthDay` based on this month-day with the requested month, not null

Throws:

`DateTimeException` - if the month-of-year value is invalid

with

`public MonthDay with(Month month)`

Returns a copy of this `MonthDay` with the month-of-year altered.

This returns a month-day with the specified month. If the day-of-month is invalid for the specified month, the day will be adjusted to the last valid day-of-month.

This instance is immutable and unaffected by this method call.

Parameters:

`month` - the month-of-year to set in the returned month-day, not null

Returns:

a `MonthDay` based on this month-day with the requested month, not null

withDayOfMonth

`public MonthDay withDayOfMonth(int dayOfMonth)`

Returns a copy of this `MonthDay` with the day-of-month altered.

This returns a month-day with the specified day-of-month. If the day-of-month is invalid for the month, an exception is thrown.

This instance is immutable and unaffected by this method call.

Parameters:

`dayOfMonth` - the day-of-month to set in the return month-day, from 1 to 31

Returns:

a `MonthDay` based on this month-day with the requested day, not null

Throws:

`DateTimeException` - if the day-of-month value is invalid, or if the day-of-month is invalid for the month

query

```
public <R> R query(TemporalQuery<R> query)
```

Queries this month-day using the specified query.

This queries this month-day using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

`R` - the type of the result

Parameters:

`query` - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

`DateTimeException` - if unable to query (defined by the query)

`ArithmeticException` - if numeric overflow occurs (defined by the query)

adjustInto

```
public Temporal adjustInto(Temporal temporal)
```

Adjusts the specified temporal object to have this month-day.

This returns a temporal object of the same observable type as the input with the month and day-of-month changed to be the same as this.

The adjustment is equivalent to using `Temporal.with(TemporalField, long)` twice, passing `ChronoField.MONTH_OF_YEAR` and `ChronoField.DAY_OF_MONTH` as the fields. If the specified temporal object does not use the ISO calendar system then a `DateTimeException` is thrown.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisMonthDay.adjustInto(temporal);
temporal = temporal.with(thisMonthDay);
```

This instance is immutable and unaffected by this method call.

Specified by:

`adjustInto` in interface `TemporalAdjuster`

Parameters:

`temporal` - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

format

`public String format(DateTimeFormatter formatter)`

Formats this month-day using the specified formatter.

This month-day will be passed to the formatter to produce a string.

Parameters:

`formatter` - the formatter to use, not null

Returns:

the formatted month-day string, not null

Throws:

`DateTimeException` - if an error occurs during printing

atYear

`public LocalDate atYear(int year)`

Combines this month-day with a year to create a `LocalDate`.

This returns a `LocalDate` formed from this month-day and the specified year.

A month-day of February 29th will be adjusted to February 28th in the resulting date if the year is not a leap year.

This instance is immutable and unaffected by this method call.

Parameters:

`year` - the year to use, from MIN_YEAR to MAX_YEAR

Returns:

the local date formed from this month-day and the specified year, not null

Throws:

`DateTimeException` - if the year is outside the valid range of years

compareTo

```
public int compareTo(MonthDay other)
```

Compares this month-day to another month-day.

The comparison is based first on value of the month, then on the value of the day. It is "consistent with equals", as defined by `Comparable`.

Specified by:

`compareTo` in interface `Comparable`<`MonthDay`>

Parameters:

`other` - the other month-day to compare to, not null

Returns:

the comparator value, that is less than zero if this is before `other`, zero if they are equal, greater than zero if this is after `other`

See Also:

`isBefore(MonthDay)`, `isAfter(MonthDay)`

isAfter

```
public boolean isAfter(MonthDay other)
```

Checks if this month-day is after the specified month-day.

Parameters:

`other` - the other month-day to compare to, not null

Returns:

true if this is after the specified month-day

isBefore

`public boolean isBefore(MonthDay other)`

Checks if this month-day is before the specified month-day.

Parameters:

`other` - the other month-day to compare to, not null

Returns:

true if this point is before the specified month-day

equals

`public boolean equals(Object obj)`

Checks if this month-day is equal to another month-day.

The comparison is based on the time-line position of the month-day within a year.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other month-day

See Also:

`Object.hashCode()`, `HashMap`

hashCode

`public int hashCode()`

A hash code for this month-day.

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
public String toString()
```

Outputs this month-day as a `String`, such as `--12-03`.

The output will be in the format `--MM-dd`:

Overrides:

`toString` in class `Object`

Returns:

a string representation of this month-day, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.